Informe técnico — Segunda entrega (Unidades 4 a 8)

Proyecto: Food Store · **Materia:** Base de Datos II — UTN FRM **Equipo G:** Ivana Gutiérrez, Rayssa Alejo, Julieta Chávez, Brisa Muñoz **Entrega:** segunda entrega del TPI · **Fecha:** noviembre de 2026 **Repositorio:** `Food Store` (rama `main`)

1. Resumen ejecutivo

Esta entrega cubre las Unidades 4 a 8 sobre el mismo modelo relacional de Food Store. Cada unidad agrega una técnica distinta —normalización, seguridad, mantenimiento, NoSQL y operación desconectada— y en todas se aplicó la misma regla de trabajo: **la IA propone, el motor y las mediciones deciden**. Ninguna cifra de este informe proviene de una estimación: todas salen de un script ejecutado y de una salida conservada en el repositorio.

Unidad	Pieza entregada	Resultado verificable
U4	FNBC sobre `control_lote_almacen` + cubo `fact_venta_categoria_dia`	Esquema en 3FN que **no** cumple FNBC; descomposición **sin pérdida** (corregida tras el hallazgo F-U4-4, con contraejemplo de la variante descartada); reporte de **138,0 ms a 0,118 ms**
U5	Roles, auditoría, simulacro de incidente, anonimización	4 roles de grupo + 2 logins + `rol_auditoria`; permiso por columna; 20.000 filas anonimizadas con equivalencia agregada exacta
U6	Respaldo, restauración, health check y mantenimiento	Respaldo de **32,62 KB**; RTO **6,18 s**; RPO **6 h 00 m 26 s**; 0 inconsistencias tras restore
U7	Migración documental y consultas cruzadas	10 pedidos migrados; 4 consultas Mongo con equivalentes SQL; paridad **7/7** y **4/4**
U8	Tablet SQLite + semáforo contra instantánea	50.006 productos; 6 solicitudes resueltas; **0 escrituras** en PostgreSQL

La entrega se completa con la declaración de uso de IA (`docs/DUIA.md`), la bitácora de prompts y correcciones (`BITACORA.md`), las especificaciones de Kiro de U4, U5 y U8, el checklist de objetivos y el code review de todos los scripts.

2. Alcance, método y condiciones de reproducción

2.1 Condiciones

• Todo se ejecutó sobre **copias de trabajo** (`food_store_tp5`, `food_store_tp6`, `foodstore_u8`, `foodstore_restore_test`), nunca sobre la base principal.

• Los scripts son autocontenidos: `schema.sql` + `seed.sql` reconstruyen el modelo y cada unidad agrega su pieza con `DROP IF EXISTS` previo.

• Las mediciones de volumen (549.529 filas en `detalle_pedido`, 227.009 pedidos) corresponden a la base masiva de TP3/TP5. Reproducir desde `seed.sql` da un conjunto mínimo: los planes son correctos, las cifras de filas no son comparables. Se documenta en cada caso.

2.2 Criterio de evidencia

Se aceptó como válido únicamente lo que cumple las tres condiciones:

1. Se ejecutó en el motor (no se leyó de una respuesta de IA). 2. Dejó una salida reproducible en el repositorio (`.txt`, `.pdf` o bloque comentado del script). 3. Tiene una verificación de contraste: `EXCEPT` bidireccional, hash, conteo antes/después o plan de ejecución.

3. Unidad 4 — FNBC y desnormalización controlada

3.1 El caso que separa 3FN de FNBC

`control_lote_almacen (LoteID, DepositoID, ResponsableControlID)` cumple dos dependencias funcionales:

• `{LoteID, DepositoID} -> ResponsableControlID` (FD1)

• `ResponsableControlID -> DepositoID` (FD2)

Las claves candidatas son **K1 = {LoteID, DepositoID}** y **K2 = {LoteID, ResponsableControlID}**. Los tres atributos son primos.

Como no hay atributos no primos, **3FN no detecta nada**. Pero FD2 tiene un determinante que no es superclave (`{ResponsableControlID}+` no contiene `LoteID`), así que **el esquema viola FNBC**. Es el caso canónico donde las dos definiciones se separan y donde citar 3FN como si fuera FNBC sería un error.

3.2 Descomposición sin pérdida y su verificación

El script descompone en `responsable_deposito (responsable_control_id PK, deposito_id)` y `control_lote (lote_id, responsable_control_id)`, ambas en FNBC. La reunión es **sin pérdida** porque el atributo común es `{ResponsableControlID}`, que es la clave primaria de `responsable_deposito` (y por FD2 determina toda esa relación): la reunión no puede multiplicar filas —cada responsable aparece una sola vez— ni perderlas —cada par lote-responsable

recupera su depósito—. El script lo verifica con `EXCEPT` bidireccional contra la vista de compatibilidad (0 filas en ambos sentidos), cardinalidad, firma y prueba inyectiva sobre `(lote_id, deposito_id)`. La lógica de la corrección fue validada con una simulación paso a paso; la reejecución de la Parte 1 en el motor (`psql -f TP6/tp_fnbc_control_lote.sql`) quedó pendiente por no disponer de contraseña en la sesión de cierre.

Hallazgo del code review (F-U4-4, severidad alta) y corrección. La primera versión usaba `control_lote (lote_id, deposito_id)` y reunía por `DepositoID`. También quedaba en FNBC, pero `{DepositoID}+ = {DepositoID}` no es superclave de ninguna de las dos relaciones: esa reunión es **con pérdida** —con dos lotes y dos responsables en el depósito 30, la vista devolvía 4 filas donde el original tiene 2—. Su `EXCEPT` 0/0 pasaba únicamente porque la instancia de ejemplo tiene exactamente un responsable por depósito. La corrección cambia `control_lote` a `(lote_id, responsable_control_id)`, agrega al script el contraejemplo ejecutable de la variante descartada y declara el costo: la FNBC no preserva la DF `{LoteID, DepositoID} -> ResponsableControlID` como restricción de esquema, de modo que la consulta inyectiva sobre la vista queda como detector de violaciones. El detalle está en `SEGUNDA_ENTREGA/code_review_segunda_entrega.md`.

Las tres anomalías (inserción, actualización, borrado) se demuestran con SQL dentro de `SAVEPOINT`s y se revierten con `ROLLBACK TO`, de modo que el esquema de prueba nunca queda inconsistente. La migración se ejecuta dentro de la transacción del script (reversible con el plan de `docs/kiro_u4/tasks.md`) y aborta si los datos heredados violan FD2, en lugar de resolver en silencio con `DISTINCT`.

3.3 Desnormalización medida, no intuida

El reporte «top 5 categorías del día» se resolvía con 4 tablas y un `Parallel Seq Scan` sobre `detalle_pedido`: **549.529 filas leídas, 5.004 buffers, 138,0 ms**. El motor leía medio millón de filas para responder sobre ~31.372 líneas del día.

Se agregó `fact_venta_categoria_dia` con grano `(fecha, categoria_id)`:

Antes	Después
`Parallel Seq Scan` · 549.529 filas · 5.004 buffers	`Bitmap Index Scan` + `Heap Scan` · 3 filas · 6 buffers
138,0 ms (233,2 ms en frío)	**0,118 ms**

El cubo se mantiene en la **misma transacción** que la escritura de origen: camino caliente O(1) por línea (`trg_detalle_fact_ins/del/upd`) y camino frío con recálculo del día para cambios de ancho (`trg_pedido_fact_upd`, `trg_producto_fact_cat`, `trg_categoria_fact_nombre`). La regla de negocio es idéntica a la consulta original: una línea cuenta si su pedido no está `CANCELADO`.

Dos desvíos del enunciado, documentados: el esquema real no tiene `eliminado` ni `subtotal` (la baja es el ENUM y el subtotal es `cantidad * precio_unitario`), y `fecha = CURRENT_DATE` sobre `TIMESTAMPTZ` devuelve 0 filas — se usa rango semiabierto. Sin esa corrección, el `EXPLAIN` mediría un conjunto vacío.

3.4 Reversibilidad

• Parte 2: `DROP TABLE fact_venta_categoria_dia CASCADE;`. Los disparadores caen con la tabla y la fuente de verdad no se toca. Reaplicar = volver a ejecutar el script (`DROP` + backfill).

• Parte 1: se borran `control_lote`, `responsable_deposito` y la vista. La tabla original `control_lote_almacen` se conserva como oráculo.

El runbook completo está en `docs/kiro_u4/tasks.md` §4.

4. Unidad 5 — Seguridad, auditoría y anonimización

4.1 Roles y mínimo privilegio

Rol	Tipo	Alcance
`rol_app_lectura`	Grupo	`SELECT` en tablas operativas y **permiso por columna** sobre `usuario`
`rol_app_escritura`	Grupo	`SELECT`/`INSERT`/`UPDATE` operativos, sin `DELETE` general
`rol_soporte`	Grupo	Operaciones acotadas de soporte
`rol_reportes`	Grupo	Solo lectura agregada; **no lee PII**
`app_web`	Login	Hereda los roles de aplicación
`admin_datos`	Login	Administración de datos
`rol_auditoria`	Grupo	Acceso separado para auditoría

La auditoría de la IA propuso `GRANT ALL`, `SELECT` sobre todas las tablas y `DELETE` «para completar el CRUD». Se rechazó: se mantuvo el permiso por columna, el *default deny* y `pg_default_acl` en 0 filas (todo privilegio es explícito). Evidencia en `docs/verificacion_permisos.txt`.

4.2 Auditoría nativa

Se activaron `log_connections`, `log_disconnections`, `log_statement` y `log_line_prefix` con `ALTER SYSTEM` + `pg_reload_conf()`, sin reinicio. Se capturó una secuencia de lectura, escritura y un login fallido:

• Fragmento crudo de **2.261 bytes**.

• SHA-256 `9C2C3C0131782C96A7E026241F2F5B183EBADB1F2ECB634F92C7FBCEA96E6C93`.

• `pg_hba.conf` quedó en `scram-sha-256` para loopback; la regla temporal `trust` usada en la prueba fue revertida y documentada.

Se distingue explícitamente lo que la auditoría nativa **no** hace frente a `pgaudit`: no registra fila por fila ni objeto por sentencia. El log se trata como activo sensible (ACL de Windows, sincronización con OneDrive).

4.3 Simulacro de incidente

El análisis forense se hizo con **solo el log anonimizado** (`docs/log_simulacro_anonimizado.txt`). La IA acertó la cronología y el escalamiento parcial, pero inventó tres cosas: que `fn_autenticar` existía (el log decía «no existe la función»), que `app_web` y `admin_datos` existían (el `GRANT` falló porque no existían) y que la sesión era de un rol de aplicación cuando en realidad todo corrió como superusuario `postgres` por `trust` desde loopback. El contraste con el log crudo y con `\du`/`\df`/`\dt` permitió separar lo leído de lo inferido. La contención la decidió el equipo con esa evidencia, no la IA.

4.4 Anonimización y equivalencia

Se construyó `usuario_anon` con datos sintéticos deterministas (sin `random()`, `now()` ni hash del dato real), mismo dominio del ENUM `rol`, `created_at` truncado a mes y `REVOKE ALL FROM PUBLIC`.

Control	Resultado
Filas	20.000 en `usuario` y 20.000 en `usuario_anon`
Distribución observada	20.000 `USUARIO`, 0 `ADMIN` (la IA había supuesto 10.000/10.000)
Equivalencia agregada (`EXCEPT` ida y vuelta)	0 filas
Control por `rol, (id % 10)`	0 filas (el mes es degenerado: un solo valor, 2026-09)
No-fuga en `mail`, `nombre+apellido`, `celular`, `contrasena`	0 coincidencias
Idempotencia	md5 `703bd7edc6d5523f3093595e59ef90eb` idéntico al reejecutar

Es un reemplazo determinista con utilidad analítica preservada: **no** es k-anonimidad ni anonimización diferencial. `id` y el mes quedan como cuasi-identificadores y la equivalencia es solo agregada.

5. Unidad 6 — Mantenimiento, respaldo y recuperación

Laboratorio sobre `foodStore`, con restauración descartable en `foodstore_restore_test`. La base principal nunca se elimina ni se reemplaza.

Evidencia	Resultado
Respaldo `pg_dump -Fc`	`foodStore_2026-10-30_172759.dump`, **32,62 KB**
Health check	Conexiones, bloqueos y ratio commits/rollbacks dentro del umbral
Mantenimiento	`VACUUM (VERBOSE, ANALYZE)`; `n_dead_tup = 0`; sin `VACUUM FULL`
Restauración	**RTO 6,18 s** (meta < 300 s) · **RPO 6 h 00 m 26 s** (meta < 24 h)
Validación	4/5/12/11/20 filas restauradas; **0 inconsistencias lógicas**

La política anti-`VACUUM FULL` es explícita: bloquea la tabla y la reescribe; el mantenimiento se hace con `VACUUM (VERBOSE, ANALYZE)`. El incidente del simulacro se usó como caso de recuperación: se midió cuánto se pierde con el respaldo disponible y se verificó que el RTO/RPO declarados se cumplen.

Hallazgo del code review: `pg_restore --no-owner --no-privileges` restaura los datos pero deja `app_web` sin `SELECT` sobre `pedido` en la base restaurada (`has_table_privilege = f`). Se corrigió el runbook: después del restore se reaplican roles y `GRANT`s.

6. Unidad 7 — NoSQL: migración documental y consultas cruzadas

6.1 Modelo y migración

Se migraron **10 pedidos** del modelo relacional a la colección `pedidos` de MongoDB con **embedding**: el pedido absorbe `cliente` y el arreglo `lineas`, congelando `precio_unitario` y `subtotal` como snapshot histórico. Se definieron 5 índices:

• `idx_pedido_id_unique` (único, sobre `pedido_id`)

• `idx_estado`

• `idx_multikey_lineas_producto` (multikey)

• `idx_total`

• `idx_cliente_id`

La colección tiene un validador `$jsonSchema` que exige `pedido_id`, `fecha`, `estado`, `forma_pago`, `cliente`, `lineas` y `total`, con ENUM para estado y forma de pago.

6.2 Consultas y paridad

Consulta	MongoDB	PostgreSQL	¿Coinciden los IDs?
4.a listar pedidos	10 documentos	10 filas	Sí
4.b `estado = TERMINADO`	7	7	Sí — `[2, 3, 4, 6, 7, 9, 10]`
4.c producto 1 en `lineas`	4	4	Sí — `[1, 4, 7, 10]`
4.d `total > 8000`	5	5	Sí — `[10, 9, 4, 7, 1]`

La diferencia de mecanismo es real: MongoDB resuelve 4.c recorriendo el arreglo embebido con el índice multikey y devuelve el documento raíz; PostgreSQL necesita reunión y deduplicación (`DISTINCT` o `EXISTS`). La paridad de resultados es del 100 % **sobre el catálogo con el que se ejecutó**.

6.3 Dictamen

Se desaconseja migrar el núcleo transaccional a MongoDB. El ciclo de un pedido exige ACID multi-documento (pedido + stock + pago) y restricciones declarativas que PostgreSQL impone en el motor. La recomendación es una arquitectura **híbrida**: PostgreSQL como *single source of truth* transaccional y MongoDB como almacén de lectura desnormalizado, replicado asincrónicamente.

6.4 Hallazgo de reproducibilidad (code review U7)

El script `consultas_postgresql.sql` usa `usuario`, `pedido.usuario_id`, `pedido.total` y el estado `'TERMINADO'`. Ninguno de los cuatro existe en el `schema.sql` de este repositorio: la entidad es `cliente` (con `cliente_id`), el total es derivable (`SUM(cantidad * precio_unitario)`) y el ENUM es `PENDIENTE | EN_PREPARACION | ENTREGADO | CANCELADO`. El script se ejecutó contra la base de trabajo de U7, cuyo catálogo difiere del publicado, de modo que la paridad 7/7 y 4/4 **no es reproducible con** `schema.sql` **+** `seed.sql` sin adaptar esos identificadores.

Decisión: se conserva el script original (fidelidad a lo ejecutado), se documenta la diferencia y se agrega `Trabajo Practico - Unidad 7/consultas_postgresql_corregidas.sql`, que usa el catálogo real, calcula el total y devuelve —esperado, derivado de las 5 líneas del seed— 1 fila por consulta (pedido 1, $3.250,00; producto 1 en el pedido 1). El detalle está en `SEGUNDA_ENTREGA/code_review_segunda_entrega.md`.

7. Unidad 8 — Tablet SQLite y semáforo

La tablet trabaja sin señal: guarda un catálogo en SQLite, arma borradores y los somete a un semáforo contra una instantánea de PostgreSQL. **El móvil propone, el servidor dispone.**

Evidencia	Resultado
Instantánea	**50.006 productos**, 2,96 MB, `fecha_copia` 2026-11-12
Catálogo disponible	50.005 (un producto con baja lógica)
C1 / C7	Catálogo sin servidor; `COMMIT` deja 1 borrador, `ROLLBACK` 0; cantidad 0 y producto inexistente rechazados por `CHECK` y FK
C2	`CONFLICTO` por precio; al aceptar, nueva solicitud `SIMULADO_APROBADO`
C3	`RECHAZADO` por stock insuficiente (500 > 106)
C4	`RECHAZADO` por baja lógica; la fila de caché queda con `eliminado = 1`
C5	`SIMULADO_APROBADO` sin cambios
C6	UUID repetido: error de PK; repetir el semáforo no reevalúa (`evaluado_en` intacto)
Integridad	`PRAGMA integrity_check` = `ok`
Servidor	Pedidos 227.009 y stock 4.989.376 sin cambios; solo baja lógica del producto 3

Decisiones técnicas que la IA propuso mal y se corrigieron:

• de `precio REAL` a `precio_centavos INTEGER` en tablas `STRICT`, porque el punto flotante no representa centavos de forma exacta.

• Borrar la fila de caché cuando el servidor da de baja un producto: **baja lógica local** (`eliminado = 1`), para no romper la FK del borrador ni perder el motivo del rechazo.

• Actualizar todas las solicitudes al reevaluar: solo `PENDIENTE`, para que el semáforo sea idempotente.

• Booleanos `0/1` al importar el CSV: psql exporta `t/f`; se conservan como texto y se normalizan con `CASE`.

8. Trazabilidad del uso de IA

Unidad	Herramienta	Qué se usó	Corrección humana relevante
U4	Kiro + OpenCode	Especificación, DDL y planes	Fecha como rango; no almacenar `total`; no MV

Unidad	Herramienta	Qué se usó	Corrección humana relevante
U5	Kiro + OpenCode	Roles, auditoría, forense, anonimización	Rechazo de `GRANT ALL`; distribución inventada reemplazada por la real
U6	OpenCode	SPEC, scripts de backup/restore, health check	ACL post-restore; sin `VACUUM FULL`
U7	OpenCode	Modelo documental, migración, consultas, dictamen	Afirmaciones de la IA refutadas con documentación oficial
U8	Kiro + OpenCode (Plan)	Esquema SQLite, semáforo, code review	Centavos, `STRICT`, lápida local, idempotencia

La declaración formal está en `docs/DUIA.md`; el detalle de prompts, propuestas y descartes, en `BITACORA.md`.

9. Code review de los scripts

Se revisaron los scripts de las cinco unidades con una escala de severidad (Alta / Media / Baja) y una decisión por hallazgo: **corregido**, **mitigado** o **documentado**. El informe completo, con el código corregido, está en `SEGUNDA_ENTREGA/code_review_segunda_entrega.md`.

Unidad	Hallazgo	Severidad	Decisión
U4	`fecha = CURRENT_DATE` devuelve 0 filas sobre `TIMESTAMPTZ`	Alta	Corregido: rango semiabierto
U4	Descomposición `(Resp, Dep) + (Lote, Dep)` con pérdida; EXCEPT 0/0 solo por la instancia	Alta	Corregido: `(Lote, Resp)` + contraejemplo
U4	Cubo sin política de retención	Baja	Documentado
U5	Propuesta de `GRANT ALL` y `DELETE` general	Alta	Rechazado; default deny
U5	Distribución de roles supuesta (10.000/10.000)	Alta	Corregido con la salida real
U6	`--no-privileges` deja `app_web` sin acceso	Alta	Corregido en el runbook
U7	SQL equivalente con catálogo inexistente (`usuario`, `total`, `TERMINADO`)	Alta	Documentado + script corregido
U7	`insertMany` sin transacción y `drop()` previo	Media	Documentado; patrón recomendado
U7	PII en el código fuente de la migración	Media	Documentado; datos sintéticos recomendados
U7	Dinero en `double` en el validador	Baja	Documentado
U8	`precio REAL`, caché con borrado físico, reevaluación total	Alta	Corregido (C1–C7)

10. Cómo reproducir la entrega

```
$env:PGPASSWORD = ''
$psql = 'C:\Program Files\PostgreSQL\18\bin\psql.exe'

# Modelo base sobre una copia
& $psql -U postgres -h localhost -d food_store_dev -f schema.sql
& $psql -U postgres -h localhost -d food_store_dev -f seed.sql

# U4 — FNBC y desnormalización
& $psql -d food_store_tp6 -v ON_ERROR_STOP=1 -f TP6/carga_dia_actual.sql
& $psql -d food_store_tp6 -v ON_ERROR_STOP=1 -f TP6/tp_fnbc_control_lote.sql
& $psql -d food_store_tp6 -v ON_ERROR_STOP=1 -f TP6/tp_desnormalizacion_top_categorias.sql

# U5 — roles, auditoría y anonimización
& $psql -d foodstore_test -v ON_ERROR_STOP=1 -f sql/roles.sql
& $psql -d foodstore_test -v ON_ERROR_STOP=1 -f sql/usuario_anon.sql
& $psql -d foodstore_test -v ON_ERROR_STOP=1 -f sql/simulacro_incidente.sql

# U6 — respaldo, health check, mantenimiento, restore y programacion
.\TP6\backup_foodstore.ps1
.\TP6\schedule_u6.ps1            # registra tareas diarias/semanales (Admin, requiere pgpass.conf)
& $psql -d foodStore -f TP6/sql/health_check.sql
& $psql -d foodStore -f TP6/sql/maintenance.sql
.\TP6\restore_test.ps1

# U7 — migración y consultas
mongosh "mongodb://localhost:27017/foodstore" "Trabajo Practico - Unidad 7/migracion_mongodb.js"
mongosh "mongodb://localhost:27017/foodstore" "Trabajo Practico - Unidad 7/consultas_mongodb.js"

# U8 — instantánea, tablet y semáforo
& $psql -d foodstore_u8 -f movil/v_snapshot_movil.sql
& $psql -d foodstore_u8 -f movil/exportar_snapshot.sql
```

```
& $psql -d foodstore_u8 -f movil/verificacion_servidor.sql
```

En SQLite: ejecutar `movil/schema_sqlite.sql`, importar `movil/servidor_snapshot.csv` en `snapshot_servidor` y ejecutar `movil/semaforo.sql` y `movil/pruebas_sqlite.sql`.

11. Riesgos y deuda técnica

Riesgo	Impacto	Mitigación
U4: la corrección F-U4-4 aún no se reejecutó en el motor	La salida verificada de la Parte 1 no está refrescada	Reejecutar `TP6/tp_fnbc_control_lote.sql` sobre `food_store_tp6` antes de la defensa
U7 no reproducible con el `schema.sql` publicado	Un tercero no puede repetir la paridad 7/7	Script corregido + nota en el informe U7
PII embebida en `migracion_mongodb.js`	Datos personales en el repositorio	Reemplazar por datos sintéticos antes de publicar
`insertMany` sin transacción	Migración parcial ante un fallo	Carga a colección de staging + `renameCollection`
Respaldo en OneDrive	Copia sincronizada, no aislada	Respaldo local + verificación de restauración (U6)
`rol_auditoria` sin `pgaudit`	Auditoría a nivel sentencia, no fila	Documentado como límite de la solución nativa
Volumen del cubo U4	Crecimiento diario × categoría	Deuda de retención para U6
ACL permisivas del log de auditoría	Lectura no autorizada del log	Documentado como deuda de hardening

12. Anexo — Mapa de archivos de la entrega

Documento	Ubicación
Informe técnico único (U4–U8)	`SEGUNDA_ENTREGA/informe_tecnico_segunda_entrega.md` / `.pdf`
Checklist de objetivos	`SEGUNDA_ENTREGA/checklist_segunda_entrega.md`
Code review de scripts	`SEGUNDA_ENTREGA/code_review_segunda_entrega.md`
DUIA consolidada	`docs/DUIA.md`
Bitácora de IA	`BITACORA.md`
Specs Kiro U4	`docs/kiro_u4/`
Specs Kiro U5	`docs/kiro_roles_spec/`
Specs Kiro U8	`docs/kiro/`
U4 — scripts e informe	`TP6/`
U5 — roles, auditoría, anonimización	`sql/`, `docs/`
U6 — respaldo y mantenimiento	`TP6/` (`backup_foodstore.ps1`, `schedule_u6.ps1`, `restore_test.ps1`, `SPEC_U6.md`), `TP6/sql/`
U7 — migración y consultas	`Trabajo Practico - Unidad 7/`
U8 — tablet y semáforo	`movil/`, `docs/`